```
$ gcc -nostartfiles pgm3.c
/usr/bin/ld: warning: cannot find entry symbol _start; defaulting to 0000000000000390
$ ./a.out

Hello World
```

Figure 5: Output of the program *pgm3.c*

```
$ gcc pgm4.c
$ ./a.out

Time taken = 14.848001 milliseconds
$ gcc -O1 pgm4.c
$ ./a.out

Time taken = 1.661000 milliseconds
$ gcc -O2 pgm4.c
$ ./a.out

Time taken = 0.004000 milliseconds
$ gcc -O3 pgm4.c
$ ./a.out

Time taken = 0.003000 milliseconds
```

Figure 6: Execution time with different levels of optimisation

```
$ gcc pgm5.c
$ ./a.out
I am always printed
$ gcc -D DB pgm5.c
$ ./a.out
I am always printed
Why I am printed now?
```

Figure 7: Conditional execution of code

the code. Some of the options include *-O0, -O1, O2, -O3, -Ofast, -Os*, etc. Consider the program *pgm4.c* to better understand optimisation with gcc. The program *pgm4.c* measures and prints the execution time taken to initialise an integer array *a[1000][1000]* with the value 1. The program is suitable for optimisation because of a feature of the memory called spatial locality. Spatial locality refers to the use of data elements within relatively close storage locations. The C programming language uses row major ordering where two-dimensional arrays are stored, row by row, in memory. However, in the program *pgm4.c*, the two nested *for* loops and the line of code *a[j][i]=1;* store the number 1 in the array column by column. Therefore the time taken to access array elements is more than the time taken to access the same number of array elements row by row.

We will use different optimisation options to achieve faster and faster execution. Incidentally, even without any compiler optimisation, we can reduce the execution time by replacing the line of code *a[j][i]=1;* with the line *a[i][j]=1;*, where the initialisation will be done row by row.

```
#include<stdio.h>
#include<time.h>
int main( )
{
    float t1,t2,t3;
    int i,j,a[1000][1000];
    t1=clock( );
    for(i=0;i<1000;i++)
    {
     for(j=0;j<1000;j++)
     {
         a[j][i]=1;
     }
    }
    t2=clock( );
    t3=(float)(((t2-t1)/CLOCKS_PER_
SEC)*1000);
    printf("\nTime taken = %f
milliseconds\n",t3);
    return 0;
}
```

The GCC manual tells us that the options *-O1, -O2*, and *-O3* optimise the code in such a way that the execution becomes faster and faster. Figure 6 shows the output of the program with and without optimisation. We can see that the execution becomes faster and faster, as expected, with the unoptimised code being the slowest and the code optimised with option *-O3* being the fastest.

Options for conditional execution

There are many options provided by gcc for efficient debugging of code. One technique is to execute a certain piece of code only if a particular macro is defined. This code can be used only for testing or debugging and will not be executed during actual runs. The option *-D* defines a macro for a preprocessor to detect. The program *pgm5.c* illustrates the use of option *-D* for the conditional execution of code.

```
#include<stdio.h>
int main()
{
    printf("I am always printed\n");
    #ifdef DB
     printf("Why I am printed now?\n");
    #endif
    return 0;
}
```

The command *gcc pgm5.c* compiles the program without the macro DB defined. The command *gcc -D DB pgm5.c* compiles the program with the macro DB defined. Figure 7 shows the output of the program *pgm5.c* with and without defining the macro DB. It can be seen from the figure that the conditionally executed line of code *printf ("Why I am printed now?\n");* gets executed only when the macro DB is defined.

Please note that what we have seen in this article is just the tip of the iceberg. There are still hundreds of options in gcc remaining to be explored. Programmers can achieve different goals like platform-independent software development, efficient software development cycles, better debugging techniques, better training methods, etc, with these options. So, I am sure it will be immensely rewarding if you dig deeper into what gcc offers. END

By: Deepu Benson

The author is a free software enthusiast whose area of interest is theoretical computer science. The open source tools of his choice include ns-2 and ns-3. He maintains a technical blog at *www.computingforbeginners.blogspot.in*.